

LAPORAN PRAKTIKUM PBO 4

Inheritance, Generalization, dan Overriding

Nama	Ananda Firdaus
NIM	251511003
GitHub	https://github.com/FirdausSinau/PBO/tree/main/Praktikum%204%20-%20Inheritance

BAGIAN A. CIRCLE DAN CYLINDER

A.1 Tujuan

Praktikum I membangun Circle lalu Cylinder secara bertahap untuk mengamati extends, constructor chain (urutan pemanggilan constructor dari subclass ke superclass), method warisan, overriding, @Override, dan super.method(). Bentuk geometrinya bukan fokus.

A.2 Kode Inti

Circle.java (superclass)

```
java
```

```
public class Circle {  
  
    private double radius;  
  
    private String color;  
  
  
    public Circle() {  
  
        radius = 1.0;  
  
        color = "red";  
  
    }  
  
  
    public Circle(double radius) {  
  
        this.radius = radius;  
  
        color = "red";  
  
    }  
  
  
    public Circle(double radius, String color) {  
  
        this.radius = radius;  
  
        this.color = color;  
  
    }  
  
  
    public double getRadius() { return radius; }  
}
```

```

    public void setRadius(double radius) { this.radius = radius; }

    public String getColor() { return color; }

    public void setColor(String color) { this.color = color; }


    public double getArea() { return radius * radius * Math.PI; }


    @Override

    public String toString() {

        return "Circle[radius=" + radius + ", color=" + color + "];"

    }
}

Cylinder.java (subclass)

java

public class Cylinder extends Circle {

    private double height;


    public Cylinder() {

        super();

        height = 1.0;

    }


    public Cylinder(double height) {

        super();

        this.height = height;

    }


    public Cylinder(double radius, double height) {

        super(radius);

        this.height = height;

    }
}

```

```

    public double getHeight() { return height; }

    public void setHeight(double height) { this.height = height; }

    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * height
            + 2 * Math.PI * getRadius() * getRadius();
    }

    public double getVolume() {
        return super.getArea() * height;
    }

    @Override
    public String toString() {
        return "Cylinder: subclass of " + super.toString() + ", height=" + height;
    }
}

```

Cylinder tidak mendeklarasikan radius maupun color, tetapi `c3.getRadius()` tetap bisa dipanggil. Field-nya ikut terwariskan dan tetap `private` milik `Circle`.

A.3 Output Program

Output `TestCircle` (Step 1 dan 2):

a: radius=1.0 area=3.141592653589793

b: radius=2.5 area=19.634954084936208

a: Circle[radius=1.0, color=red]

b: Circle[radius=2.5, color=red]

-- sebelum setter --

c: radius=3.0 area=28.274333882308138 color=blue

-- sesudah setter --

c: radius=5.0 area=78.53981633974483 color=green

Objek a dan b memakai `getArea()` yang sama, tetapi hasilnya berbeda karena radius kedua objek berbeda.

Output `TestCylinder` (Step 5 dan 9):

c1

radius = 1.0

height = 1.0

base area = 12.566370614359172

volume = 3.141592653589793

c2

radius = 1.0

height = 10.0

base area = 69.11503837897544

volume = 31.41592653589793

c3

radius = 2.0

height = 10.0

base area = 150.79644737231007

volume = 125.66370614359172

Cylinder: subclass of Circle[radius=2.0, color=red], height=10.0

Baris "base area" berisi luas permukaan, bukan luas alas (c3: $48\pi = 150.796$). Output ini diambil setelah `getArea()` di-override pada Step 8.

Pembagian method (Checkpoint 3):

Method	Ditulis di Cylinder?	Asal
constructor, getHeight, setHeight, getVolume	Ya	Cylinder
getRadius, setRadius	Tidak	Warisan Circle
getColor, setColor	Tidak	Warisan Circle

new Cylinder(10.0) memberi nilai 10.0 ke height, sedangkan radius tetap 1.0. Pemanggil bisa mengira 10.0 adalah radius. Ini saya catat sebagai temuan dan tidak saya ubah di modul ini.

A.4 Bukti Constructor Chain (Step 6)

Urutan output new Cylinder(2.0, 10.0) dengan println sementara di setiap constructor:

```
>> Circle(double) dipanggil      <- bagian superclass lebih dulu
```

```
>> Cylinder(double, double) dipanggil <- baru bagian subclass
```

```
c3 radius = 2.0
```

Jawaban Checkpoint 4: bagian Circle dikerjakan lebih dulu. Baris Circle(double) tercetak sebelum baris Cylinder(double, double).

Eksperimen: super(radius) dihapus dari constructor Cylinder(double, double).

```
java
```

```
public Cylinder(double radius, double height) {
```

```
//    super(radius);    <-- DIHAPUS
```

```
    this.height = height;
```

```
}
```

Program tetap compile tanpa error, tetapi hasilnya berubah:

>> Circle() dipanggil <- berubah ke constructor tanpa argumen

>> Cylinder(double, double) dipanggil

c3 radius = 1.0 <- diharapkan 2.0

Java menyisipkan `super()` otomatis saat `super(...)` tidak ditulis, sehingga `Circle()` yang berjalan dan radius kembali ke 1.0. Nilai 2.0 masuk ke parameter radius dan tidak pernah dipakai. Compiler tidak memberi peringatan karena 2.0 sah secara tipe.

A.5 Bukti Akses `private` Ditolak (Step 7)

Method sementara di dalam `Cylinder`:

```
java
```

```
public double cobaAksesRadius() {  
    return radius;  
}
```

Pesan error compiler:

Cylinder.java:2: error: radius has private access in Circle

```
    public double cobaAksesRadius() { return radius; }  
                                ^
```

1 error

Percobaan yang sama dari sisi pemanggil (`TestCircle`, Step 2):

TestCircle.java:19: error: radius has private access in Circle

```
    a.radius = 10.0;
```

Jawaban Checkpoint 5: `extends` membuat subclass mewarisi behavior yang dapat diakses, tetapi tidak membuat field `private` superclass menjadi dapat diakses langsung. `private` berlaku per class, bukan per hierarchy, dan `Cylinder` adalah class lain. Perbaikannya `return getRadius();`, karena `getRadius()` bersifat `public` dan ikut diwarisi.

A.6 Override `getArea()`, Bug `getVolume()`, dan Perbaikannya (Step 8)

Sebelum override, `getArea()` pada `Cylinder` masih milik `Circle` dan menghasilkan luas alas ($\pi \times r^2$). `getVolume()` memanggil `getArea()` biasa. Untuk `c3` ($r = 2.0$, $h = 10.0$): `area = 12.566370614359172` dan `volume = 125.66370614359172` (benar).

Setelah `getArea()` diubah menjadi luas permukaan dan sebelum `getVolume()` diperbaiki:

```
getArea() = 150.79644737231007
```

```
getVolume() = 1507.9644737231006 (volume sebenarnya 125.66370614359172)
```

Tidak ada error dan semua angka terlihat oke, tetapi volumenya salah.

`getVolume()` memanggil `getArea()` versi `Cylinder`, bukan versi `Circle`. Java memilih method berdasarkan tipe runtime objek, yaitu tipe objek yang sebenarnya saat program berjalan. `c3` bertipe `Cylinder`, dan `Cylinder` punya `getArea()` sendiri:

```
c3.getVolume()
```

- > memanggil `getArea()`

- > tipe objek: `Cylinder`

- > `Cylinder` punya `getArea()` sendiri (hasil override)

- > yang berjalan: versi luas permukaan

- > $150.796... \times 10 = 1507.96$ (SALAH)

Perhitungannya:

salah: $(2\pi r h + 2\pi r^2) \times h = (40\pi + 8\pi) \times 10 = 480\pi = 1507.96$

benar: $\pi r^2 h = 4\pi \times 10 = 40\pi = 125.66$

rasio: $48\pi / 4\pi = 12$ (sesuai $1507.96 / 125.66$)

Perbaiki dengan `super.getArea()`:

```
java
```

```
public double getVolume() {  
    return super.getArea() * height;  
}
```

Penggunaan Super	getArea()	getVolume()
Tanpa super .	150.79644737231007	1507.9644737231006 (salah)
Dengan super .	150.79644737231007 (tidak berubah)	125.66370614359172 (benar)

getArea() yang dipanggil langsung tidak berubah setelah perbaikan, yang berubah hanya getVolume(), jadi super.method() bekerja di satu titik pemanggilan dan tidak berlaku global.

Jawaban Checkpoint 6: getArea() pada objek Cylinder sekarang berbeda dari getArea() pada objek Circle. super.getArea() dipakai ketika kode di dalam subclass dengan sengaja memakai implementasi superclass. Ada dua jenis override di sini:

Jenis	Contoh	Fungsi
Override penuh	getArea())	Mengganti makna method sepenuhnya
Override yang memakai super .	getVolume())	Memakai versi baru, tetapi tetap bisa memanggil versi lama

A.7 Override toString() dan Uji @Override (Step 9)

```
java
```

```
@Override
```

```
public String toString() {
```

```
    return "Cylinder: subclass of " + super.toString() + ", height=" + height;
```

```
}
```

Output `System.out.println(c3):`

Cylinder: subclass of Circle[radius=2.0, color=red], height=10.0

`super.toString()` menghasilkan `Circle[radius=2.0, color=red]`, sehingga format tidak perlu ditulis ulang.

Eksperimen 1: nama method diganti `toStrng()` dengan `@Override` dipertahankan.

Cylinder.java:27: error: method does not override or implement a method from a supertype

```
@Override
```

```
^
```

1 error

Eksperimen 2: `@Override` dihapus, nama tetap `toStrng()`. Tidak ada error. `toStrng()` hanya menjadi method baru yang tidak pernah dipanggil. Java mencari `toString()` di Cylinder, tidak menemukannya, lalu memakai versi Circle:

Circle[radius=2.0, color=red]

Nilai `height` hilang. Objeknya Cylinder, tetapi tercetak seperti Circle.

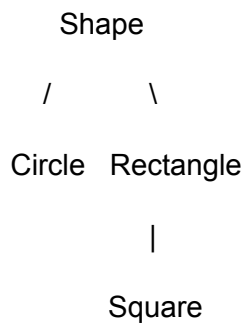
Jawaban Checkpoint 7: `@Override` tidak membuat inheritance terjadi, itu tugas `extends`. `@Override` adalah pernyataan "niat" kepada compiler bahwa method ini menggantikan method superclass. Typo langsung ditolak saat compile. Tanpa `@Override`, typo lolos dan baru terlihat dari hasil program yang salah. `@Override` tidak menambah kemampuan program, hanya mengubah "salah tapi diem diem" jadi "gagal compile".

BAGIAN B. SHAPE, CIRCLE, RECTANGLE, DAN SQUARE

B.1 Tujuan

Bagian B menentukan kapan inheritance layak dipakai. Saya mencari sendiri class yang pantas menjadi superclass, dan hasilnya dipakai di Bagian C.

B.2 Diagram Hierarchy



B.3 Generalisasi: Apa yang Naik ke Superclass

Circle dan Rectangle sama-sama punya color dan filled. Keduanya dipindah ke Shape supaya tidak ditulis dua kali.

Aturan yang saya pakai: hanya state atau behavior yang identik di semua subclass yang naik ke atas, yang hanya "hampir sama" tetap di subclass.

State / Behavior	Identik?	Ditaruh di
color, setColor(x) hanya menyimpan nilai	Ya	Shape
filled, setFilled(b) hanya menyimpan nilai	Ya	Shape
radius, width, length	Tidak semua class punya	Subclass

<code>getArea(): $\pi \times r^2$ vs $w \times l$</code>	Rumus berbeda	Subclass
<code>getPerimeter(): $2\pi r$ vs $2(w + l)$</code>	Rumus berbeda	Subclass
<code>getSide()</code>	Hanya Square	Square

Aturan yang sama berlaku untuk method. `getArea()` yang dipaksa masuk Shape membuat setiap subclass wajib meng-override-nya, dan subclass yang lupa akan memakai rumus bentuk lain. Polanya sama dengan bug `getVolume()` di A.6.

B.4 Kode Inti

Shape.java

```
java
```

```
public class Shape {

    private String color;

    private boolean filled;

    public Shape() {

        color = "green";

        filled = true;

    }

    public Shape(String color, boolean filled) {

        this.color = color;

        this.filled = filled;

    }

    public String getColor() { return color; }
```

```

    public void setColor(String color) { this.color = color; }

    public boolean isFilled() { return filled; }

    public void setFilled(boolean filled) { this.filled = filled;
}

@Override

    public String toString() {

        return "Shape[color=" + color + ", filled=" + filled +
    "]" + ";

    }

}

```

Circle.java (Bagian B)

java

```

public class Circle extends Shape {

    private double radius;

    public Circle() {

        super();

        radius = 1.0;

    }

    public Circle(double radius) {

        super();

        this.radius = radius;

    }

    public Circle(double radius, String color, boolean filled) {

```

```

        super(color, filled);

        this.radius = radius;
    }

    public double getRadius() { return radius; }

    public void setRadius(double radius) { this.radius = radius; }

    public double getArea() { return Math.PI * radius * radius; }

    public double getPerimeter() { return 2 * Math.PI * radius; }

    @Override
    public String toString() {
        return "Circle[radius=" + radius + ", " + super.toString()
+ " ]";
    }
}

```

File ini tidak berisi `getColor()` maupun `setColor()`, tetapi keduanya tetap bisa dipanggil. Jadi generalisasinya berhasil: `color` ditulis satu kali di `Shape` dan dipakai semua bentuk. Cara mengeceknya: ubah `Shape.getColor()` menjadi `return "MERAH";`, lalu keempat objek ikut berubah.

Square.java

```
java
```

```

public class Square extends Rectangle {

    public Square() {

        super(1.0, 1.0);

    }
}

```

```
public Square(double side) {  
    super(side, side);  
}
```

```
public Square(double side, String color, boolean filled) {  
    super(side, side, color, filled);  
}
```

```
public double getSide() { return getWidth(); }
```

```
public void setSide(double side) {  
    super.setWidth(side);  
    super.setLength(side);  
}
```

```
@Override
```

```
public void setWidth(double side) {  
    super.setWidth(side);  
    super.setLength(side);  
}
```

```
@Override
```

```
public void setLength(double side) {  
    super.setWidth(side);  
    super.setLength(side);  
}
```

```

@Override

public String toString() {

            return "Square[side=" + getSide() + ", " +
super.toString() + "];

    }

}

```

Square tidak punya field baru. Ia memakai width dan length milik Rectangle dengan satu aturan tambahan: width == length.

B.5 Output Hierarchy Utuh (Step 16)

Shape[color=black, filled=false]

Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172

Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0

Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]] area=25.0

Baris Square memperlihatkan tiga level super.toString() yang berantai. color=green, filled=true hanya ditulis satu kali, di Shape.

B.6 Bukti Square Rusak dan Perbaikannya (Step 14 dan 15)

Sebelum setter di-override:

setWidth(8) -> width=8.0, length=5.0, area=40.0, perimeter=26.0

setLength(3) -> width=8.0, length=3.0, area=24.0, perimeter=22.0

Setelah setWidth(8), objek berukuran 8 x 5 dan sudah bukan persegi. Compiler tidak memberi peringatan, dan semua angka benar secara hitungan untuk bentuk yang salah.

Setelah setter di-override:

setWidth(8) -> width=8.0, length=8.0, area=64.0, perimeter=32.0

setLength(3) -> width=3.0, length=3.0, area=9.0, perimeter=12.0

Kedua sisi selalu sama. Aturan width == length disebut invarian, yaitu kondisi yang harus selalu benar untuk sebuah objek, dan di sini invarian itu hadir/terjaga atau apalah itu.

Jawaban Checkpoint 11: setelah setWidth(8.0), objek tidak lagi merepresentasikan Square. Inheritance memberi behavior yang tersedia secara teknis tetapi belum tentu

sesuai aturan subclass, dan override dipakai untuk menjaga aturan itu. Rectangle tidak salah, karena `setWidth()` sudah benar untuk Rectangle. Aturan `width == length` hanya diketahui Square, jadi Square yang harus menyesuaikan.

Jawaban Checkpoint 12: `setWidth()` dan `setLength()` harus di-override karena keduanya adalah pintu masuk data yang bisa merusak invarian. `getArea()` dan `getPerimeter()` boleh memakai versi Rectangle karena hanya membaca nilai, dan rumusnya tetap benar saat `width == length`. Buktinya ada di output: `setWidth(8)` menghasilkan `area=64.0` (8×8) dan `perimeter=32.0` ($2 \times (8 + 8)$) tanpa override apa pun.

Prinsipnya: override ketika behavior warisan melanggar invarian subclass, dan jangan override ketika behavior warisan sudah benar.

`getArea()` di Square juga tidak perlu di-override menjadi `side * side`. Berdasarkan analisis kode, cara itu hanya menyembunyikan masalah. `setWidth(8.0)` tetap menghasilkan objek `8 x 5`, `getArea()` melaporkan `64.0`, dan `getPerimeter()` warisan tetap melaporkan `26.0`. Dua method pada objek yang sama jadi saling bertentangan, dan itu lebih sulit dilacak. Perbaikan harus dilakukan di sumbernya, yaitu setter. (Bagian ini hasil analisis kode, bukan hasil run.)

BAGIAN C. PENGEMBANGAN KASUS TUGAS SEBELUMNYA

C.1 Kasus yang Dipilih

Kasus yang dikembangkan adalah tugas mandiri "Inventaris Sarana dan Prasarana" yang diberikan pada Rabu pagi, bukan kasus Week 3. Jenis barang yang berbeda punya laju penyusutan nilai yang berbeda, jadi hierarchy-nya tumbuh dari model dan tidak dibuat-buat.

C.2 Diagram Sebelum dan Sesudah

Sebelum (tanpa inheritance):

Barang

Lokasi

PencatatanInventaris

Tiga class berdiri sendiri dan tidak punya hubungan tipe.

Sesudah:

Barang

/ \

Elektronik Perabot

Barang <--- komposisi (has-a) --- PencatatanInventaris

Lokasi

C.3 Kandidat Generalisasi (Step 17)

Superclass	Subclass kandidat	State / behavior yang mirip	Kalimat "is-a"
Barang	Elektronik	kode, nama, status, hargaBeli, umurBulan,	"Elektronik adalah sebuah Barang" (lolos)

		tandaiRusak(), perbaiki()	
Barang	Perabot	sama persis	"Perabot adalah sebuah Barang" (lolos)
Lokasi	RuangKelas, Laboratorium	kodeLokasi, namaLokasi	masuk akal, tetapi tidak dipakai karena di luar cakupan
Barang	PencatatanInven- taris	tidak ada	"PencatatanInventaris adalah sebuah Barang" (gagal)

PencatatanInventaris tidak dibuat extends Barang karena ia memiliki barang (has-a), bukan adalah barang (is-a). Hubungan has-a disebut komposisi: satu class menyimpan objek class lain sebagai field. Modul menyebut inheritance jangan dipakai kalau kalimat is-a kerasa gak masuk akal.

C.4 Alasan Memakai Inheritance

Elektronik dan Perabot tanpa behavior yang berbeda hanya menjadi pemegang data tambahan (merek, garansi, bahan), dan ga cukup untuk membenarkan tiga file. Saya menambahkan getNilaiTaksir() dengan tarif penyusutan yang berbeda per jenis:

Class	Penyusutan per bulan	Alasan
Barang	5%	tarif umum
Elektronik	10%	komponen cepat aus
Perabot	2%	tahan lama

Hierarchy ini punya dua manfaat. Pertama, `getNilaiTaksir()` di-override sehingga tiap jenis punya tarif sendiri. Kedua, semua barang bisa diproses seragam lewat array `Barang[]` tanpa `if`.

C.5 Kode: `extends`, `super(...)`, `@Override`, `super.method()`

`Barang.java` (superclass)

```
java
```

```
package model;
```

```
public class Barang {
```

```
    private String kodeBarang;
```

```
    private String namaBarang;
```

```
    private String statusKondisi;
```

```
    private double hargaBeli;
```

```
    private int umurBulan;
```

```
    public Barang(String kodeBarang, String namaBarang, String  
statusKondisi) {
```

```
        this(kodeBarang, namaBarang, statusKondisi, 0.0, 0);
```

```
    }
```

```
    public Barang(String kodeBarang, String namaBarang, String  
statusKondisi,
```

```
        double hargaBeli, int umurBulan) {
```

```
        this.kodeBarang = kodeBarang;
```

```
        this.namaBarang = namaBarang;
```

```
        this.statusKondisi = statusKondisi;
```

```
        this.hargaBeli = hargaBeli;
```

```
        this.umurBulan = umurBulan;
```

```

    }

    public String getKodeBarang() { return kodeBarang; }

    public String getNamaBarang() { return namaBarang; }

    public String getStatusKondisi() { return statusKondisi; }

    public double getHargaBeli() { return hargaBeli; }

    public int getUmurBulan() { return umurBulan; }

    public void tandaiRusak() { this.statusKondisi = "rusak"; }

    public void tandaiHilang() { this.statusKondisi = "hilang"; }

    public void perbaiki() { this.statusKondisi = "bagus"; }

    public double getNilaiTaksir() {
        return Math.max(0.0, hargaBeli * (1 - 0.05 * umurBulan));
    }

    public String getJenis() { return "Barang"; }

    public static String formatRupiah(double nilai) {
        return "Rp" + String.format("%.0f", nilai);
    }

    @Override
    public String toString() {
        return getKodeBarang() + " " + getNamaBarang()
            + " [" + getStatusKondisi() + ", " +
            formatRupiah(getHargaBeli()) + " ]";
    }

```

```
    }  
}
```

Constructor 3 parameter di Barang memakai `this(...)`, yaitu memanggil constructor lain di class yang sama, bukan constructor induk. Gunanya menghindari lima baris assignment yang sama ditulis dua kali. `super(...)` dipakai di Elektronik dan Perabot untuk memanggil constructor induk.

Elektronik.java (subclass 1)

```
java
```

```
package model;
```

```
public class Elektronik extends Barang {  
    private String merek;  
    private int garansiBulan;  
  
    public Elektronik(String kodeBarang, String namaBarang, String  
statusKondisi,  
                        double hargaBeli, int umurBulan, String  
merek, int garansiBulan) {  
        super(kodeBarang, namaBarang, statusKondisi, hargaBeli,  
umurBulan);  
        this.merek = merek;  
        this.garansiBulan = garansiBulan;  
    }  
  
    public String getMerek() { return merek; }  
    public int getGaransiBulan() { return garansiBulan; }  
  
    @Override  
    public double getNilaiTaksir() {
```

```
        return Math.max(0.0, getHargaBeli() * (1 - 0.10 *
getUmurBulan()));
    }
```

```
@Override
```

```
public String getJenis() { return "Elektronik"; }
```

```
public double getHargaJualPasar() {
```

```
    return super.getNilaiTaksir() * 0.9;
```

```
}
```

```
public boolean masihDigaransi() {
```

```
    return getUmurBulan() <= garansiBulan;
```

```
}
```

```
@Override
```

```
public String toString() {
```

```
    return getJenis() + " " + super.toString()
```

```
        + " (" + merek + ", garansi " + garansiBulan + "
b1n"
```

```
        + (masihDigaransi() ? ", aktif" : ", habis") +
")";
```

```
}
```

```
}
```

```
Perabot.java (subclass 2)
```

```
java
```

```
package model;
```

```

public class Perabot extends Barang {

    private String bahan;


    public Perabot(String kodeBarang, String namaBarang, String
statusKondisi,

                        double hargaBeli, int umurBulan, String bahan)
{

    super(kodeBarang, namaBarang, statusKondisi, hargaBeli,
umurBulan);

    this.bahan = bahan;

}


    public String getBahan() { return bahan; }


    @Override

    public double getNilaiTaksir() {

        return Math.max(0.0, getHargaBeli() * (1 - 0.02 *
getUmurBulan()));

    }


    @Override

    public String getJenis() { return "Perabot"; }


    @Override

    public String toString() {

        return getJenis() + " " + super.toString() + " (" + bahan
+ ")";

    }

}

```

Perabot tidak menulis ulang `perbaiki()` karena behavior warisannya sudah benar. Menulis ulang hanya jadi duplikasi, mirip mirip Square yang tidak perlu meng-override `getArea()`.

`PencatatanInventaris.java` (komposisi, tidak diubah)

```
java
```

```
public String getRingkasan() {  
  
    return kodePencatatan + " -> " + barang  
  
        + " di " + lokasi.getKodeLokasi() + " " +  
    lokasi.getNamaLokasi();  
  
}
```

`getRingkasan()` memanggil `barang.toString()` tanpa `if`. Versi yang tampil (Elektronik, Perabot, atau Barang) mengikuti tipe runtime objeknya.

C.6 Output Pengujian

=== DATA PENCATATAN INVENTARIS ===

Kode pencatatan: PCT001

Barang: 01 - Laptop

Status: rusak

Lokasi: GDG001 - Gudang

Kode pencatatan: PCT002

Barang: 02 - Proyektor

Status: bagus

Lokasi: KLS001 - Ruang Kelas 1

=== PERUBAHAN STATUS BARANG ===

Status laptop sebelum diperbaiki: rusak

Status laptop sesudah diperbaiki: bagus

=== SUBCLASS DIBUAT DARI SUPERCLASS ===

dari Barang : B01 / Laptop ASUS / rusak

dari Elektronik : ASUS / 24 bln

dari Perabot : Kayu

=== OVERRIDE getNilaiTaksir() ===

Laptop (Elektronik, 10%/bln): Rp3,000,000

Proyektor(Elektronik, 10%/bln): Rp2,400,000

Meja (Perabot, 2%/bln): Rp1,900,000

Kursi (Perabot, 2%/bln): Rp210,000

-- dipanggil lewat tipe Barang[] --

Elektronik -> Rp3,000,000

Elektronik -> Rp2,400,000

Perabot -> Rp1,900,000

Perabot -> Rp210,000

=== super.getNilaiTaksir() DI DALAM OVERRIDE ===

Laptop (harga beli Rp15,000,000, umur 8 bln)

getNilaiTaksir() : Rp3,000,000 versi Elektronik (10%/bln)

pembanding Barang : Rp9,000,000 versi Barang (5%/bln), override tidak dipakai

getHargaJualPasar() : Rp8,100,000 memakai super.getNilaiTaksir() * 0.9

garansi 24 bln, umur 8 bln -> AKTIF

garansi 6 bln, umur 8 bln -> HABIS

=== METHOD WARISAN DARI SUBCLASS ===

laptop -> B01 Laptop ASUS [rusak]

setelah tandaiRusak() : rusak

setelah perbaiki() : bagus

(tandaiRusak dan perbaiki tidak ditulis di Elektronik)

=== ENCAPSULATION ===

statusKondisi di Barang private dan tanpa setter.

Satu-satunya jalan: tandaiRusak, tandaiHilang, perbaiki.

kursi sekarang: rusak

setelah tandaiHilang(): hilang

=== KOMPOSISI: PencatatanInventaris (has-a) ===

PCT010 -> Elektronik B01 Laptop ASUS [bagus, Rp15,000,000] (ASUS, garansi 24 bln, aktif) di KLS001 Ruang Kelas 1

PCT011 -> Perabot B03 Meja Belajar [bagus, Rp2,500,000] (Kayu) di GDG001 Gudang

=== RINGKASAN OBJECT ===

Elektronik B01 Laptop ASUS [bagus, Rp15,000,000] (ASUS, garansi 24 bln, aktif)

Elektronik B02 Proyektor Epson [bagus, Rp12,000,000] (Epson, garansi 6 bln, habis)

Perabot B03 Meja Belajar [bagus, Rp2,500,000] (Kayu)

Perabot B04 Kursi Plastik [hilang, Rp350,000] (Plastik)

C.7 Bukti super.method() (Eksperimen T4-04)

Dengan super.getNilaiTaksir() (versi benar):

getNilaiTaksir() : Rp3,000,000

getHargaJualPasar() : Rp8,100,000

Tanpa super.(getNilaiTaksir()) biasa, sehingga versi Elektronik ikut terpakai):

pembandingan Barang : Rp9,000,000

getHargaJualPasar() : Rp2,700,000

Perhitungannya:

dengan super -> tarif 5% : $15.000.000 \times (1 - 0.05 \times 8) = 9.000.000$

$9.000.000 \times 0.9 = \text{Rp}8.100.000$ (benar)

tanpa super -> tarif 10% : $15.000.000 \times (1 - 0.10 \times 8) = 3.000.000$

$3.000.000 \times 0.9 = \text{Rp}2.700.000$ (salah)

Selisihnya Rp5.400.000. Program tidak error dan angkanya tercetak rapi, tetapi harga jual pasar tercatat terlalu rendah. Kesalahan seperti ini baru ketahuan saat barang dijual atau diaudit. Polanya sama dengan bug `getVolume()` di Bagian A.

BAGIAN D. TABEL SKENARIO UJI T4-01 SAMPAI T4-07

ID	Skenario	Expected Result	Bukti	Status
T4-01	Objek subclass dibuat lewat constructor	Bagian superclass dan subclass terinisialisasi benar	Objek yang sama menampilkan B01 / Laptop ASUS / rusak (diisi super(...) ke Barang) dan ASUS / 24 bln (diisi this.merek di Elektronik)	PASS
T4-02	Method warisan dipanggil dari subclass	Behavior superclass tersedia tanpa copy code	<code>laptop.tandaiRusak()</code> menghasilkan rusak, <code>laptop.perbaiki()</code> menghasilkan bagus. Kedua method hanya ada di <code>Barang.java</code> , tidak di <code>Elektronik.java</code>	PASS
T4-03	Method override dipanggil	Behavior subclass yang berjalan	Lewat loop <code>Barang[]</code> : Laptop menghasilkan Rp3.000.000 (10%/bln), Meja Rp1.900.000 (2%/bln). Nama method sama, hasil mengikuti tipe runtime	PASS
T4-04	Override memakai <code>super.method()</code>	Sebagian behavior superclass tetap dipakai	<code>getHargaJualPasar()</code> = Rp8.100.000. Tanpa <code>super</code> . hasilnya Rp2.700.000 (eksperimen C.7)	PASS
T4-05	Akses state mengikuti encapsulation	Field private tidak diakses langsung	<code>statusKondisi</code> bersifat private tanpa setter. Status kursi berubah dari rusak ke hilang hanya lewat <code>tandaiHilang()</code> . Error <code>radius</code> has private access in Circle	PASS

			(A.5) menunjukkan field private tetap tertutup dari subclass	
T4-06	Dua subclass diuji	Perbedaan behavior terlihat di output	Dua objek Elektronik dan dua objek Perabot. Tarif penyusutan berbeda (10% vs 2%) dan toString() berbeda ((ASUS, garansi 24 bln, aktif) vs (Kayu))	PASS
T4-07	Regression tugas lama	Fungsi utama sebelumnya tetap berjalan	Bagian awal Main sama dengan versi asli: status laptop rusak lalu bagus, dua pencatatan dan dua lokasi tercetak sama. Constructor 3 parameter Barang dipertahankan	PASS

BAGIAN E. DEFINITION OF DONE

#	Pernyataan	Status
1	Semua Step 1 sampai 16 sudah dicoba, bukan hanya kode final yang disalin	Done. Termasuk eksperimen hapus <code>super(radius)</code> , akses <code>private</code> , typo <code>@Override</code> , dan <code>Square.setWidth(8)</code>
2	Output dan minimal satu error eksperimen tersimpan sebagai bukti	Done. Ada 2 pesan error compiler (A.5, A.7) dan 4 silent bug (A.4, A.6, B.6, C.7)
3	Bisa menunjukkan method yang diwarisi dan yang dioverride	Done (A.3, C.6)
4	Bisa menjelaskan urutan constructor dan fungsi <code>super(...)</code>	Done (A.4)
5	Bisa menjelaskan mengapa <code>super.getArea()</code> diperlukan di <code>getVolume()</code>	Done (A.6)
6	Bisa menjelaskan mengapa <code>Square</code> perlu override setter	Done (B.6)
7	Kasus tugas sebelumnya sudah direfactor dan lulus skenario uji utama	Done. 7 dari 7 T4 PASS (Bagian C dan D)
8	PDF dan source siap di GitHub	Done

BAGIAN F. SELF ASSESSMENT

Pernyataan	Skor 1-5
Saya dapat menjelaskan extends dari kode yang saya jalankan.	3
Saya dapat membedakan inherited method dan overridden method.	3
Saya memahami mengapa constructor subclass memanggil <code>super(...)</code> .	2
Saya dapat menggunakan <code>super.method(...)</code> dengan alasan yang tepat.	3
Saya dapat menjaga encapsulation pada hierarchy class.	3
Saya dapat menentukan kapan inheritance masuk akal pada kasus sendiri.	4

BAGIAN G. REFLEKSI

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?

Circle hanya berisi radius dan color, Cylinder juga. Cylinder hanya menambah height. Kalau radius dan color ditulis ulang di Cylinder, data yang sama punya dua tempat, dan jika ada perubahan di satu tempat mudah terlupa di tempat lain.

2. Berdasarkan Step 5, apa arti praktis extends bagi objek Cylinder?

Cylinder bisa melakukan semua yang bisa dilakukan Circle tanpa menulis ulang. `c3.getRadius()` dan `c3.getColor()` berjalan walau gak ada di `Cylinder.java`. Perubahan di Circle ikut berlaku ke Cylinder, jadi hanya ada satu sumber saja.

3. Mengapa constructor Circle dijalankan ketika Anda membuat objek Cylinder?

Objek Cylinder memiliki bagian Circle di dalamnya, dan bagian itu harus terisi lebih dulu sebelum bagian Cylinder.

4. Mengapa `getVolume()` menjadi salah setelah `getArea()` dioverride?

`getVolume()` memanggil `getArea()` tanpa `super.`, dan Java memilih versi method berdasarkan tipe runtime objek. `c3` bertipe `Cylinder` yang punya `getArea()` sendiri, jadi versi luas permukaan yang dipakai. Untuk `c3`, hasilnya 12 kali lebih besar (1507.96 dibanding 125.66). Rumusnya tidak salah. Yang berubah adalah arti `getArea()` di dalam satu hierarchy, dan perubahan itu tidak terlihat saat saya menulis override-nya.

5. Apa yang secara eksplisit Anda minta ketika menulis `super.getArea()` atau `super.toString()`?

Saya meminta Java naik ke class induk dan memakai implementasi milik induk, bukan versi di class ini. Pada `getVolume()`, artinya memakai luas alas. Pada `toString()`, artinya memakai format Circle sehingga radius dan color tidak perlu ditulis ulang. Efeknya hanya berlaku di titik pemanggilan itu. `c3.getArea()` yang dipanggil dari luar tetap memakai versi `Cylinder`.

6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?

Rectangle tidak mengenal aturan persegi. Hanya Square yang tahu aturan itu, jadi Square yang harus menyesuaikan “bahan” yang disediakan Rectangle.

7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.

Tidak. Kemiripan bentuk tidak cukup, harus ada hubungan "adalah sebuah". Pada kasus inventaris, Barang dan Lokasi sama-sama punya kode dan nama, tetapi "Lokasi adalah sebuah Barang" tidak masuk akal, jadi tidak dijadikan inheritance.

8. Lengkapi: "Inheritance bukan sekadar mengurangi duplikasi, tetapi ..."

Tetapi membuat subclass ikut bertanggung jawab atas aturannya sendiri. Subclass mewarisi kode beserta resikonya. Ketika kode warisan tidak cocok dengan aturan subclass, override dipakai. Nilainya ada pada pencegahan kesalahan yang tidak terlihat: volume yang salah 12 kali lipat, Square yang bukan persegi, dan harga jual pasar yang tercatat Rp5.400.000 lebih rendah.

BAGIAN H. PENGGUNAAN AI

Ai digunakan untuk membantu Saya dalam pengerjaan praktikum 1 hingga pembuatan laporan. Saya mengimplementasikan PREDICT -> CODE -> RUN -> OBSERVE -> EXPLAIN, Ai tidak langsung memberikan Code tanpa penjelasan (langsung mengerjakan Task, kecuali yang disediakan pada Modul) melainkan meminta Saya untuk mendesain terlebih dahulu, dan melakukan sesi tanya jawab tiap Checkpoint & Task untuk membantu Saya tentang pemahaman materi. Hampir keseluruhan laporan dibuat menggunakan Ai dan berdasarkan pemahaman Saya dari chat history awal praktikum hingga akhir, dan telah Saya cek dan baca terlebih dahulu.